

Continuous Integration

Cohort 3 Team 3

Fares Almutairi

Taro Chandiwana

Lily Gaunt

Jacob Last

Alex Ponomarenko

Giulia Sclano

CI Approach Summary

With the implementation aspect of our project being coded by multiple people, we felt that it was important to practice continuous integration (CI) methods as soon as we began coding. There were multiple factors that we agreed upon that make up our CI approach. These helped us keep integration issues small, so that any problems could be resolved quickly, and as little time as possible was wasted solving such problems.

Firstly, we agreed that every time anyone works on code, they should update their own project, to ensure that it remains compatible with the main branch. This was appropriate because having five people coding and integrating their code at random times meant that the main branch would constantly be changing, and so updating regularly allowed us to not only keep integration issues small, but to also keep up with where we were at in terms of others' progress.

Another aspect of our approach was deciding that it would be best to keep how often code was integrated up to the programmer's discretion, but to have a minimum of doing it at least once every coding session. We decided that this was the best approach, as not everyone would necessarily be coding in a scheduled manner (such as daily), so giving the choice to each programmer meant that each person could make the CI most appropriate for whatever they were working on. Keeping the boundary of doing it at least once a coding session ensured that this flexibility would not be taken advantage of.

In terms of tools, we decided on using Github Actions as since we were already using Github to collaborate our code, using Github Actions would make it easy to keep up with the CI, which would be visible from the Github repository. Our workflow consisted of an automated build and test. This ensured that any compilation errors or failed tests could be identified immediately and easily, to prevent bugs in the code from crashing the program.

CI Infrastructure

As mentioned above, we used Github Actions, with a workflow which was triggered whenever anything was pushed to main or completed a pull request, to ensure that integrating code would be checked before and after merging.

We set this to run with Java version 17, with it always installing JDK 17 when running the tests to ensure that they are all run on the same Java version, to avoid inconsistencies. We also set this to run on the latest virtual machine versions of multiple operating systems (Ubuntu, Windows and MacOS), to ensure the game is widely accessible, meeting UR_SYSTEM_REQUIREMENTS.

We set up Gradle within the workflow, ensuring that it had execution permissions, so that the project could be built with the Gradle wrapper. The Gradle build and any automated tests were run, with any fail causing the whole CI job to fail.